In [1]:
```
### Training with manually updating W with "Backward" ###

import torch
#from torch.autograd import Variable
import torch.nn as nn
import torch.nn.functional as F

import torch.optim as optim




data = [(1.0,2.1,3.0), (2.0, 3.5, 6.0), (3.0, 3.0, 9.0), (4.0, 2.1, 12.0), (5.0,


class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(2,1,bias=False)  # in dim, out dim
    def forward(self, x):
        x = self.fc1(x)
        return x

net = Net()

print(net)
print(list(net.parameters()))

#input = torch.randn(1)
#out = net(input)

#def criterion(out, label):
#    return (label - out)**2
criterion = nn.MSELoss()


optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.5)
#optimizer = optim.Adam(net.parameters(), lr=0.005)

for epoch in range(20): # 0 - 19
    for i, current_data in enumerate(data):
        X1, X2, Y = current_data
        X, Y = torch.FloatTensor([X1, X2]), torch.FloatTensor([Y])
        optimizer.zero_grad()
        outputs = net(X)
        loss = criterion(outputs, Y)
        loss.backward()
        optimizer.step()    ## This line is equivalent to "W = W - lr* W.grad"
        print("Epoch {} - loss: {}".format(epoch, loss))

### Test the trained network ###
for i, current_data in enumerate(data):
    X1, X2, Y = current_data
    X, Y = torch.FloatTensor([X1, X2]), torch.FloatTensor([Y])
    out = net(torch.FloatTensor(X))
    print("when x = {}, y = {}".format(X, out))
```

```
Net(
  (fc1): Linear(in_features=2, out_features=1, bias=False)
)
[Parameter containing:
tensor([[ 0.2841, -0.3596]], requires_grad=True)]
Epoch 0 - loss: 12.047791481018066
Epoch 0 - loss: 36.4962158203125
Epoch 0 - loss: 39.242515563964844
Epoch 0 - loss: 30.495121002197266
Epoch 0 - loss: 21.62595558166504
Epoch 0 - loss: 8.914396286010742
Epoch 1 - loss: 1.3558436632156372
Epoch 1 - loss: 16.855249404907227
Epoch 1 - loss: 28.929401397705078
Epoch 1 - loss: 24.59705924987793
Epoch 1 - loss: 1.7306362390518188
Epoch 1 - loss: 14.119721412658691
Epoch 2 - loss: 0.9993358850479126
Epoch 2 - loss: 12.483170509338379
Epoch 2 - loss: 17.82200813293457
Epoch 2 - loss: 12.068835258483887
Epoch 2 - loss: 0.045652735978364944
Epoch 2 - loss: 9.560882568359375
Epoch 3 - loss: 0.5620837807655334
Epoch 3 - loss: 6.611355781555176
Epoch 3 - loss: 7.961326599121094
Epoch 3 - loss: 4.2429375648498535
Epoch 3 - loss: 0.1064584031701088
Epoch 3 - loss: 4.575018882751465
Epoch 4 - loss: 0.2299918681383133
Epoch 4 - loss: 2.541238307952881
Epoch 4 - loss: 2.5228946208953857
Epoch 4 - loss: 0.9541176557540894
Epoch 4 - loss: 0.25998449325561523
Epoch 4 - loss: 1.5728883743286133
Epoch 5 - loss: 0.06501144170761108
Epoch 5 - loss: 0.6571980714797974
Epoch 5 - loss: 0.4718724191188812
Epoch 5 - loss: 0.07363957166671753
Epoch 5 - loss: 0.24024465680122375
Epoch 5 - loss: 0.34079498052597046
Epoch 6 - loss: 0.009579538367688656
Epoch 6 - loss: 0.07779992371797562
Epoch 6 - loss: 0.01582687348127365
Epoch 6 - loss: 0.012733228504657745
Epoch 6 - loss: 0.14429791271686554
Epoch 6 - loss: 0.022933972999453545
Epoch 7 - loss: 2.1057474441477098e-05
Epoch 7 - loss: 0.0006776386871933937
Epoch 7 - loss: 0.02599821612238884
Epoch 7 - loss: 0.07200641185045242
Epoch 7 - loss: 0.06260064989328384
Epoch 7 - loss: 0.0065550291910767555
Epoch 8 - loss: 0.0015811698976904154
Epoch 8 - loss: 0.02642574906349182
Epoch 8 - loss: 0.06840488314628601
Epoch 8 - loss: 0.08142344653606415
Epoch 8 - loss: 0.019116060808300972
Epoch 8 - loss: 0.029722910374403
Epoch 9 - loss: 0.002724848221987486
```

```
Epoch 9 - loss: 0.03707887977361679
Epoch 9 - loss: 0.06465400010347366
Epoch 9 - loss: 0.05494639277458191
Epoch 9 - loss: 0.003324357559904456
Epoch 9 - loss: 0.03218075633049011
Epoch 10 - loss: 0.002229682635515928 3
Epoch 10 - loss: 0.02782193012535572
Epoch 10 - loss: 0.03937903046607971
Epoch 10 - loss: 0.02635866403579712
Epoch 10 - loss: 6.823960575275123e-05
Epoch 10 - loss: 0.021210839971899986
Epoch 11 - loss: 0.0012364145368337631
Epoch 11 - loss: 0.014498445205390453
Epoch 11 - loss: 0.017300592735409737
Epoch 11 - loss: 0.009089854545891285
Epoch 11 - loss: 0.00026994163636118174
Epoch 11 - loss: 0.009979868307709694
Epoch 12 - loss: 0.0004973221221007407
Epoch 12 - loss: 0.005474641919136047
Epoch 12 - loss: 0.005368749611079693
Epoch 12 - loss: 0.0019799417350441217
Epoch 12 - loss: 0.0006016942788846791
Epoch 12 - loss: 0.003365171840414405
Epoch 13 - loss: 0.00013725647295359522
Epoch 13 - loss: 0.0013785924529656768
Epoch 13 - loss: 0.0009640264324843884
Epoch 13 - loss: 0.0001359023153781891
Epoch 13 - loss: 0.0005400574300438166
Epoch 13 - loss: 0.0007051206193864346
Epoch 14 - loss: 1.9136285118293017e-05
Epoch 14 - loss: 0.00015179517504293472
Epoch 14 - loss: 2.4991608370328322e-05
Epoch 14 - loss: 3.6292920412961394e-05
Epoch 14 - loss: 0.0003184154920745641
Epoch 14 - loss: 4.146341234445572e-05
Epoch 15 - loss: 3.0859723665344063e-09
Epoch 15 - loss: 3.1194795155897737e-06
Epoch 15 - loss: 6.65018887957558e-05
Epoch 15 - loss: 0.00016928557306528091
Epoch 15 - loss: 0.00013581338862422854
Epoch 15 - loss: 1.805886859074235e-05
Epoch 16 - loss: 3.8642169783997815e-06
Epoch 16 - loss: 6.33211457170546e-05
Epoch 16 - loss: 0.0001586144499015063
Epoch 16 - loss: 0.00018437334802001715
Epoch 16 - loss: 4.058383638039231e-05
Epoch 16 - loss: 6.963318446651101e-05
Epoch 17 - loss: 6.249093985388754e-06
Epoch 17 - loss: 8.454528142465279e-05
Epoch 17 - loss: 0.0001454475859645754
Epoch 17 - loss: 0.00012191783753223717
Epoch 17 - loss: 6.7486107582226396e-06
Epoch 17 - loss: 7.278769044205546e-05
Epoch 18 - loss: 4.991741207049927e-06
Epoch 18 - loss: 6.205264071468264e-05
Epoch 18 - loss: 8.699171303305775e-05
Epoch 18 - loss: 5.752572906203568e-05
Epoch 18 - loss: 9.255109034711495e-08
Epoch 18 - loss: 4.704348975792527e-05
Epoch 19 - loss: 2.7188807507627644e-06
```

```
Epoch 19 - loss: 3.178304905304685e-05
Epoch 19 - loss: 3.7567886465694755e-05
Epoch 19 - loss: 1.945465919561684e-05
Epoch 19 - loss: 6.789341568946838e-07
Epoch 19 - loss: 2.1747928258264437e-05
when x = tensor([1.0000, 2.1000]), y = tensor([2.9990], grad_fn=<SqueezeBackward4
>)
when x = tensor([2.0000, 3.5000]), y = tensor([5.9981], grad_fn=<SqueezeBackward4
>)
when x = tensor([3., 3.]), y = tensor([8.9978], grad_fn=<SqueezeBackward4>)
when x = tensor([4.0000, 2.1000]), y = tensor([11.9976], grad_fn=<SqueezeBackward
4>)
when x = tensor([5.0000, 7.2000]), y = tensor([14.9957], grad_fn=<SqueezeBackward
4>)
when x = tensor([ 6.0000, 10.1000]), y = tensor([17.9945], grad_fn=<SqueezeBackwa
rd4>)
```